콜라짱·콜칩: 산책 기록에서 온디바이스 행동 추론까지

가족이 같은 반려견의 산책을 번갈아 맡으면서 시작했습니다. 앞선 산책에서 얼마나 걸었고 어디서 배변했는지 알아야 다음 산책의 시간과 동선을 정하기 쉬웠습니다. 콜라짱은 휴대폰·Wear OS 산책 기록 앱으로 구현했고, 콜칩은 그때 발견한 문제를 바탕으로 별도 저장소에서 다시 설계했습니다. 자동 행동 인식과 짧은 상대 위치 추정은 제품 기준에 도달하지 못해 중단했습니다.

콜라짱은 2025.02~2026.01, 콜칩은 2026.03~2026.08의 개발 기록입니다. Kotlin·Compose·Room·Wear Data Layer로 기기와 화면을 연결하고, 콜칩에서는 TFLite 추론과 Rust 기반 서버·분석도 실험했습니다. 기술을 추가한 순서보다, 왜 다음 구조가 필요했는지를 중심으로 정리했습니다.

1. 말로 설명하기 어려운 장소를 지도에 남기기

"그 나무 옆에서 배변했다"는 설명은 사람마다 가리키는 곳이 달랐습니다. 산책 경로 위에 시간과 위치를 연결한 마커를 남기고, 끝난 산책은 지도·달력·통계로 다시 보도록 만들었습니다. 초기 콜라짱은 휴대폰의 Room을 기준 저장소로 쓰는 로컬 앱이었고, 가족 간 클라우드 공유까지 완성한 서비스는 아니었습니다.

위치를 점으로 연결하자 정지 중에도 경로와 거리가 늘었습니다. 반대로 점을 많이 지우면 짧은 왕복과 회전이 사라졌습니다. 이후 위치 선정기에서는 정지 중 반복 관측된 위치를 기준으로 삼고, 이동 중에는 짧은 구간에서 기기가 보고한 오차 추정치가 작은 점을 고르는 방식을 구현했습니다. 원본과 표시용 경로를 구분하고 선택 이유도 남겼습니다. 지도 선을 매끄럽게 만드는 것과 실제 위치 오차를 줄이는 것은 별도 문제로 남았습니다.

2. 휴대폰을 꺼내기 어려워 손목 시계를 리모컨으로 쓰기

리드줄을 잡은 채 휴대폰을 꺼내 앱을 여는 사이, 반려견이 갑자기 줄을 당길 수 있었습니다. 사람 손목의 갤럭시 워치에서 산책을 제어하고 기록을 입력하도록 했습니다. 명령은 Wear Data Layer로 휴대폰에 전달하고, 휴대폰의 저장 상태를 시계에 돌려주는 구조였습니다. 음성 입력도 검토했지만 완성 기능으로 포함하지 않았습니다.

처음에는 목록이 바뀔 때마다 과거 기록 전체를 직렬화해 보냈습니다. 산책이 길어지면 입력 한 번에도 처리량이 커졌습니다. 이 경험은 다음 설계에서 최신 상태와 변경분을 구분하고, 전송 완료와 저장 완료를 따로 다루는 출발점이 됐습니다. 초기 앱에는 Wear 제어 일부의 미완성 경로도 남아 있었습니다.

3. 버튼 입력을 줄이려 반려견 센서와 ML0·ML1·ML2를 나누기

배변 전 탐색과 회전처럼 보호자가 익힌 패턴을 다른 보호자도 이해하게 만들고 싶었습니다. 사람 휴대폰의 센서로는 반려견의 몸짓을 측정하기 어려워 워치를 하네스에 부착했습니다. 워치는 가속도·각속도·위치를 수집하고, 휴대폰이 저장과 추론을 담당하도록 했습니다. 사람 손목의 워치와 반려견에 붙인 워치의 데이터도 구분했습니다.

ML0는 부착 방향이 달라도 움직임을 비교할 수 있게 센서 좌표를 변환하는 단계입니다. 행동을 학습하는 모델은 아니며, 실제 몸 방향을 정확히 알아냈다고 보장하지 않습니다. ML1은 같은 순간의 이동·자세·머리·코·입 등을 나눠 판단합니다. 걷다가 냄새를 맡는 복합 상태를 한 이름으로 덮지 않으려는 선택입니다. ML2는 ML1의 확률을 시간순으로 읽어 배변 전 흐름을 찾으려 했습니다. 사용자가 누른 배변 마커는 학습 정답을 연결할 때만 쓰고 예측 입력에는 넣지 않았습니다.

4. 연결이 끊겨도 다시 처리할 원본을 남기기

센서를 보냈다는 응답만 받고 워치의 원본을 지우면, 휴대폰 저장 전에 오류가 났을 때 복구할 자료가 없어집니다. 콜칩에서는 워치에 크기 제한이 있는 임시 파일 저장소를 두고, 휴대폰이 Room 저장을 끝낸 뒤 확인 응답을 보내도록 했습니다. 확인받지 못한 배치는 재전송하고, 같은 배치가 다시 와도 중복 저장하지 않도록 검사했습니다.

늦게 도착한 데이터를 도착 시각으로 라벨과 연결하면 다른 행동을 학습하게 됩니다. 따라서 산책·장치 세션·순번·센서 시각을 함께 보존했습니다. 산책 시작·종료 판단도 한 실행 주체에 모아, 화면을 다시 열거나 오래된 명령이 도착했다고 끝난 산책을 다시 수집하지 않게 했습니다. 이 구조와 테스트는 남겼지만, 모든 물리적 단절과 장시간 사용에서 무손실을 입증한 것은 아닙니다.

5. 수집·지도·추론·학습이 같은 자원을 다투는 문제 줄이기

새 위치 하나마다 전체 경로를 다시 읽거나, 여러 기능이 같은 원본을 반복 계산하면 산책이 길어질수록 작업이 커졌습니다. 표시할 경로와 과거 분석을 나누고, 화면은 필요한 범위와 미리 계산한 결과를 읽도록 바꿨습니다. 학습도 작은 작업으로 나눠 실시간 추론에 실행 기회를 주고, 열과 배터리 상태에 따라 보류하도록 했습니다. 장시간 실산책의 발열·배터리 기준을 모두 통과한 결과는 남기지 못했습니다.

이전 세대에는 Rust·LanceDB로 유사 구간 검색, DataFusion으로 집계를 실험했습니다. 과거 서버의 변경분 동기화와 CI 이미지 빌드도 구현했지만, 이 구성이 최종 콜칩의 운영 서버로 유지된 것은 아닙니다. 근거 설명과 생성 모델의 실험도 완성된 사용자 기능으로 합산하지 않았습니다.

6. 실행되는 모델과 쓸 수 있는 기능을 구분해 중단하기

사용자가 늦게 누르거나 종료를 놓친 라벨은 센서 구간의 정확한 정답이 아니었습니다. 비슷한 장면이 학습과 시험에 함께 들어가지 않도록 산책 단위로 나누고, 흔한 상태만 답하는 단순 방식과 비교했습니다. ML1은 일부 신호를 구분했지만 예측이 자주 뒤집혔고, 별도로 남긴 산책에서는 제품 가치를 확인하지 못했습니다. ML2는 독립 평가에 연결된 사건 표본이 부족했습니다.

위치도 별도였습니다. GPS 공백을 센서로 잇는 후보는 한 반려견의 과거 자료에서 단순 방식보다 나았지만, 사람과 반려견 사이 약 2m의 앞뒤 관계를 알아낸 것은 아니었습니다. 두 기기 위치의 오차가 리드줄 길이보다 클 수 있어 소프트웨어 보정만으로 목표를 충족하지 못했습니다.

2026년 8월 콜칩 개발을 중단하며 관측 위치와 사용자 확인 기록, 수집·복구 코드를 보존하고 검증되지 않은 자동 추론은 사용자 기능에서 차단했습니다. 콜칩에서는 AI 에이전트를 구현·리팩터링·테스트·리뷰에 활용했습니다. 사용자 문제, 현장 데이터 수집, 설계 경계와 평가 기준, 결과 채택·거절 및 중단 판단은 직접 맡았습니다.

설계·평가 집계·선별 소스 보기